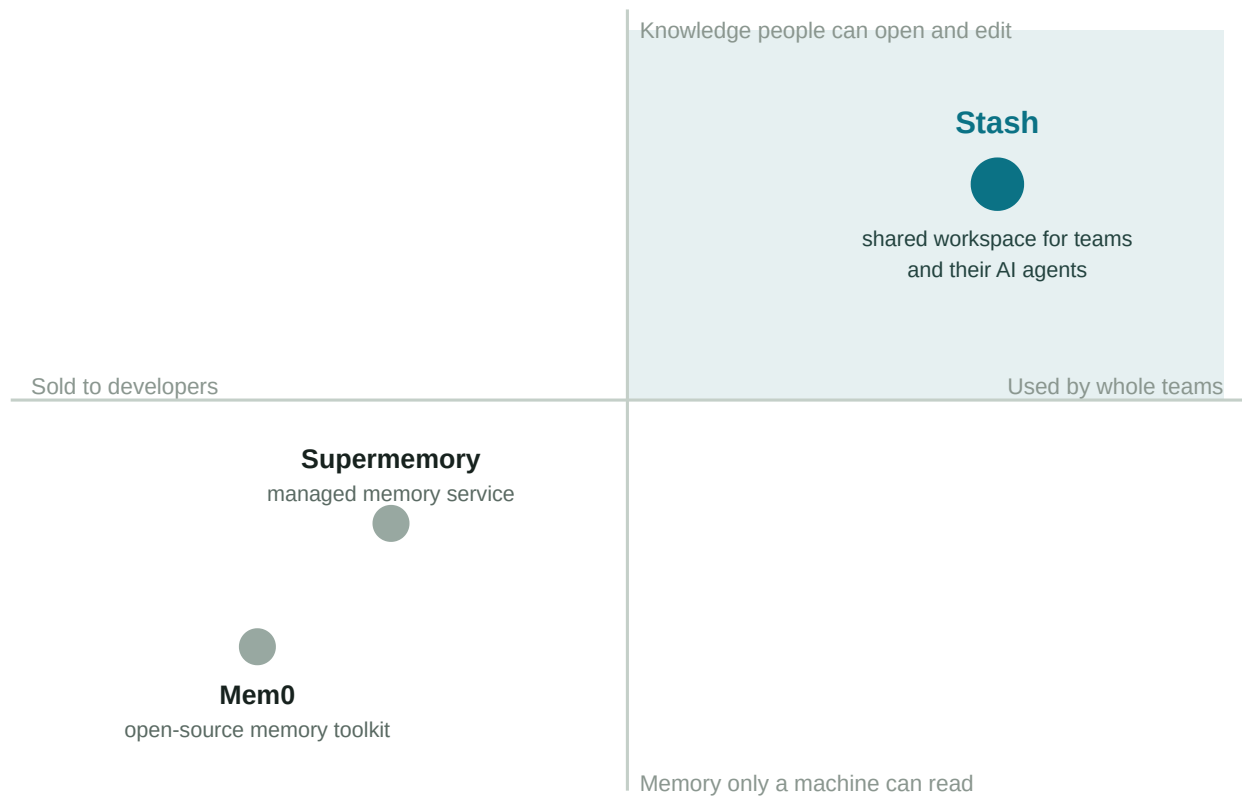


Where Stash sits in the AI memory market, and where I fit

Farzan Aman Khan · github.com/cheese-cakee · July 2026

Supermemory and Mem0 sell memory infrastructure to developers building AI apps. It lives inside someone else's product, and the end user never sees it. **Stash sells something else:** one knowledge base that a whole team and its AI agents share. Different buyer, different product, and in my view the stronger position.

The market at a glance



Stash has the top-right quadrant to itself: knowledge the whole team shares, in a form people can actually read.

Supermemory and Mem0 compete with each other, and with every AI lab that keeps adding built-in memory to its models. Stash is not in that fight. It keeps something none of them do: **the record of what a team's agents tried, what got decided, and why.**

The comparison that matters

	STASH	SUPERMEMORY	MEMO
What it really is	A shared workspace for teams and their AI agents	A memory service developers plug into their apps	An open-source memory toolkit for developers
Who pays for it	Companies whose engineers work with AI agents	App builders	App builders
What the memory looks like	Ordinary files and pages that anyone can open, check, and fix	Extracted facts, managed through an API	Extracted facts, managed through an API
Whose memory it is	The whole team's, shared	Isolated per user or container	Per user, session, or agent
What self-hosting gets you	The full product: workspace, search, agent access	A local memory API; the hosted platform is the full product	A memory server; the managed platform carries the extras

Three reasons the Stash approach wins

1 Agents already speak its language

Coding agents spend their whole day running *ls*, *find* and *grep*. Stash looks like a filesystem, so they can use it from the first minute. There is no new interface to teach and no integration project to sell.

2 Memory you can open and fix

When Supermemory or Mem0 store a wrong fact, it sits in a database until the AI gives a bad answer. In Stash a wrong memory is a file. Someone opens it, fixes it, and moves on. Companies need that kind of visibility before they will trust a memory product with real work.

3 It keeps what nobody else does

Stash records what a team's agents attempted, what worked, and what got abandoned. Ask any engineer what they most wish they knew: it is usually why the last attempt failed. The other two keep distilled facts; Stash keeps the full record.

Where I add value

Four things Stash will need as it grows, matched against work I have already shipped. Every claim below is a link you can click and check.

WHAT STASH NEEDS

Keep secrets out of shared memory

Agent transcripts carry passwords and API keys. Once they land in shared, searchable memory, one person's leak becomes everyone's leak.

Stay fast as the knowledge base grows

Search speed is the product. It has to hold up at 100× today's data.

Win the self-hosting customer

The companies that most need Stash are the ones that can't send data to a third-party cloud. Self-hosting has to feel effortless.

Security that enterprise buyers trust

Stash asks companies to store their most sensitive work product. Its own security posture is part of the pitch.

WHAT I'VE ALREADY DONE

I built a scanner for this exact problem

→ [Overtrust](#), my security tool that finds leaked secrets and credentials. The same checks belong in Stash's upload path, and I know how to put them there.

Recent performance work you can check

→ [11 merged PRs at Lucebox](#) (an AI inference startup) over the past few weeks, across cache eviction, GPU memory paging, and CI hardening.

Infrastructure work at the Linux Foundation

→ LFX mentee (picked from 151 applicants), building the [Hierio workflow app](#) end to end. Your open [issue #563](#) on self-hosted setups is where I would start.

Work merged into security-critical projects

→ Merged fixes in [Ceph](#) and [CRIU](#), and CI hardening at the Linux Foundation, where I patched real shell-injection risks.

A note on scope. These four areas are where I would start, not where I would stop. In the past nine months I went from C++ systems work to GPU inference at Lucebox to building a GitHub App for the Linux Foundation; I pick up new domains quickly and go where the roadmap needs me. Both my current commitments, the LFX mentorship and Lucebox, are going well. I am talking to Stash because this is the problem I want to work on next.